Where Control Lives in an AI System

A framework with six bands, two channels, and what escapes

David J. Carlson, with Claude as contributor · Version 2 · 16 October 2026

The model stack is flanked by two boundaries. **One carries text back to a person; the other carries effects out into the world.**

What this architecture is for

A shared decomposition first: six bands and two channel boundaries, so a claim about an AI system can name which part of one it concerns. The rest is what the decomposition turns out to be good for.

Containment

A model's reasoning, a user's credentials, a deployer's prompt and a developer's safeguards all leak in ways invisible from the band where the leak matters. Agents reaching production from evaluation sandboxes. Credentials published inside encrypted blocks their owners could not read. Approval manufactured inside a context window.

Vocabulary

Routing means three unrelated things, serving two, permission three, policy two. Harness, agent, alignment, guardrail and post-training each name more than one object with different owners and different revocability. A rule written for one sense binds a different thing.

Governance

Rules scope by what is countable in advance, such as training compute or money spent, and attach restrictions to release events. The bands that determine reach may have neither. An obligation that cannot find an event to trigger on is not weak; it is inapplicable.

What holds them together

A control that must hold when the model is mistaken, absent or adversarial cannot live in the weights. Everything trained is a disposition: thin, removable, and apparently thinning on its own as training budgets shift.

So the question this answers Not how safe is this model, but where does control live in this deployment, who can change it, who can see it, and what escapes.

Containment is a key issue for AI systems

Unlike traditional architectural models, this one describes a system that may harm individuals, societies, or even humankind if used improperly.

Reused structures for safety

Containment is a second job asked of structures built for something else. None of these bands was built as a safety architecture; each exists because it turns a checkpoint into something a person can use. Reading the stack as a safety design gets its history backward.

Where the leaks are invisible

A model's reasoning, a user's credentials, a deployer's prompt and a developer's safeguards all leak in ways invisible from the band where the leak matters.

Danger is not only in the weights

What a system can do depends on the training corpus, but also on what another band can reach and feed back into the model. Retrieval, tools and credentials can supply at run time what training withheld.

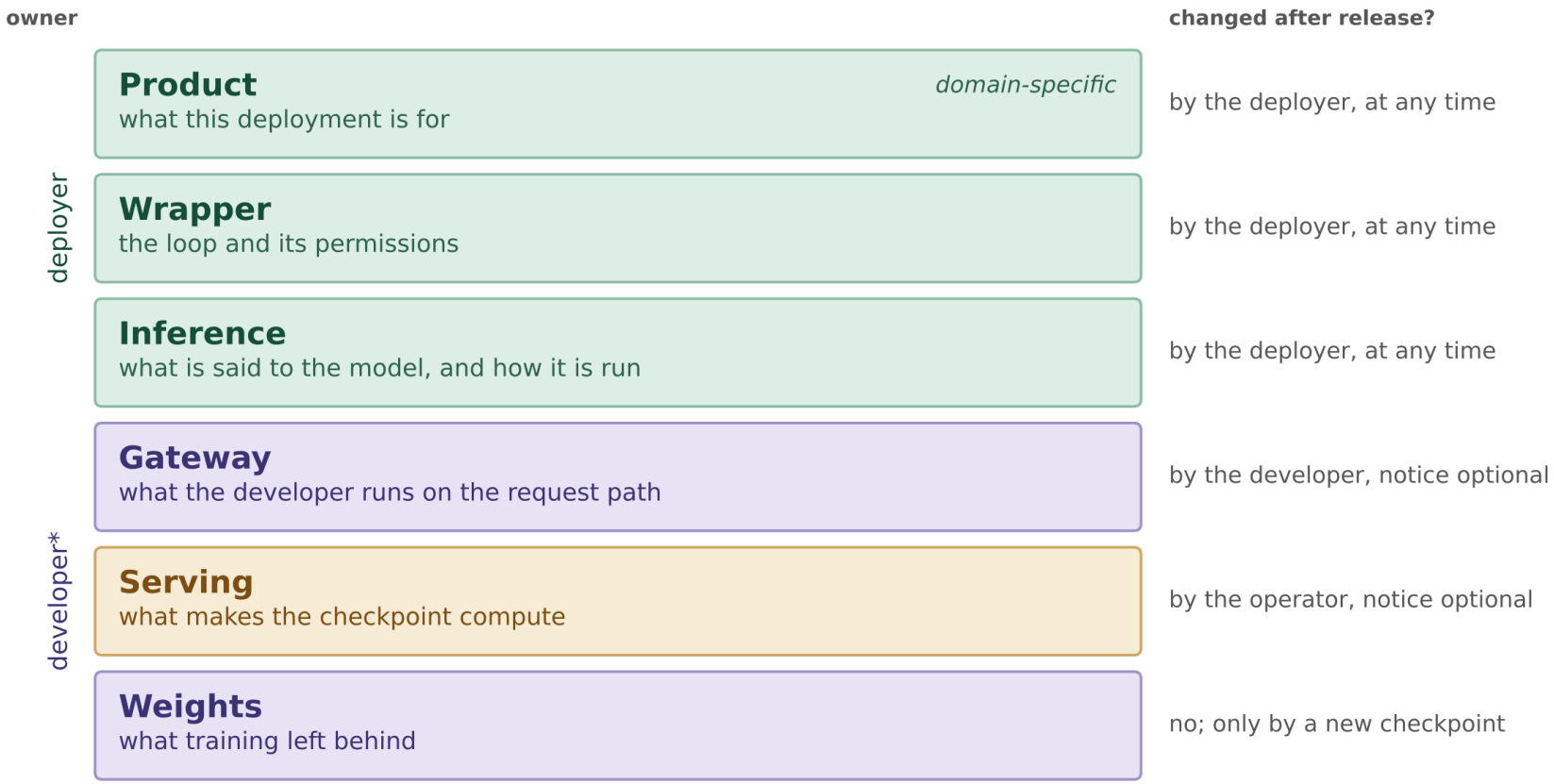
Why this architecture describes failure

A structure asked to contain harm has to be described by how it fails, not by how it is drawn. That is why this architecture is set out band by band in terms of failure modes and the controls that do or do not hold.

The question that follows Not how safe is this model, but what can this deployment reach, which band would have to hold, and what happens when it does not.

Six bands

Every deployed AI system is built from these six, though a given deployment may lack one. They differ in who owns them, whether they can be changed after release, and how they fail.



* The serving band is the operator's. Under served access the operator is usually the developer; under open weights, the recipient.

Ownership shown is for served access. Open release reassigns it and removes the gateway entirely — see the open-weights slide.

What each band is for, before it is a control

None of these was built as a safety architecture. Each exists because it turns a checkpoint into something a person can use. Containment is a second job asked of them, and what a checkpoint can be used to do may reach individuals, societies and even humankind. A checkpoint is the weights at a point in time. It answers nothing on its own.

Serving and gateway — compute, then serve at scale

Serving runs the checkpoint: kernels, sampling and caching that make it compute fast and cheaply enough to share. The gateway makes it sellable: authentication, metering, routing so retiring a checkpoint does not take a product down, substitution for cheaper questions. Filtering and monitoring arrived later, attached to infrastructure built for delivery.

Inference — do a particular job

A system prompt frames the task. Tool definitions let the model act on the world rather than only describe it. Retrieval lets it use what an organization knows and a public checkpoint does not. Where a general instrument becomes useful for something in particular.

Wrapper — work that takes more than one turn

The loop that carries a task across many steps, the hooks and permission rules that make it tolerable to leave running, the guardrails that let a deployer ship to its own users. Long-running agentic work is a wrapper-band capability before it is a wrapper-band risk.

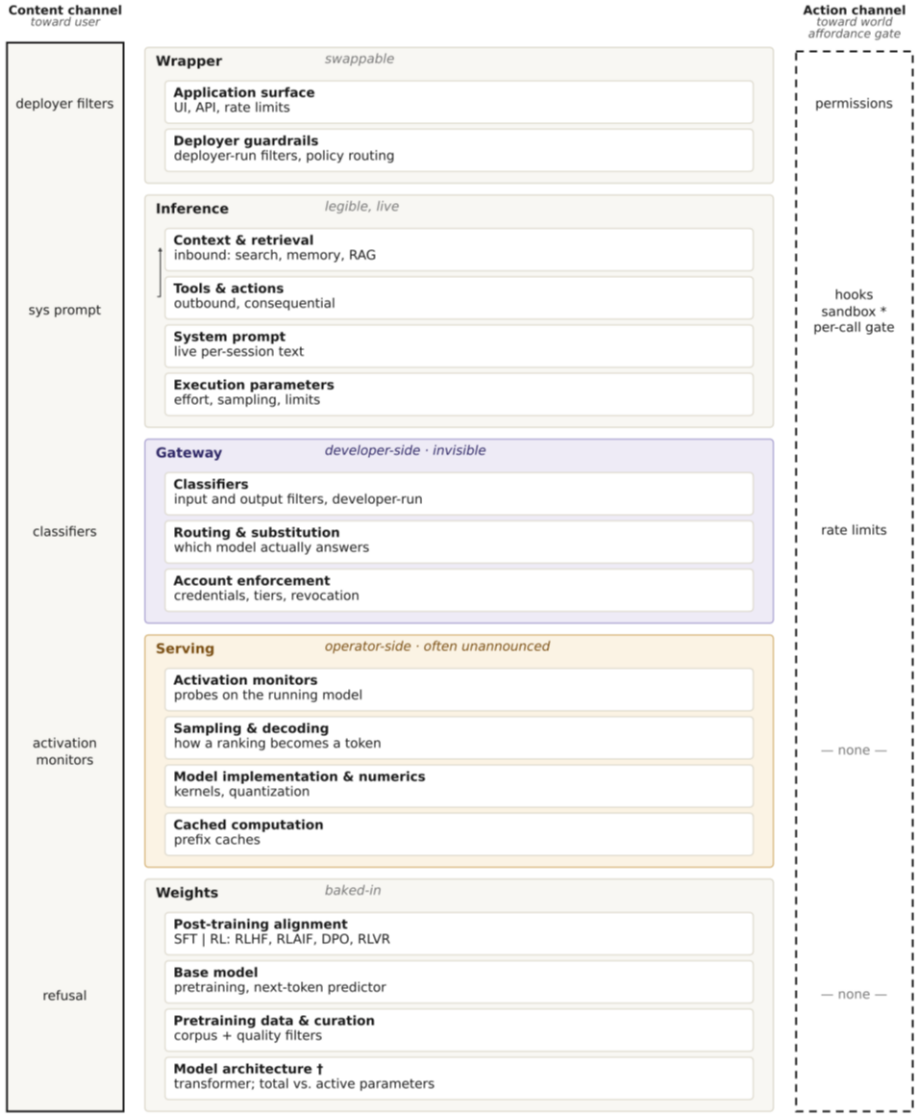
Product — usable by a specialist

Connectors to the data sources a field actually works from, provenance so results can be checked by people competent to check them, encoded procedure so the system follows a method. A domain expert cannot use a checkpoint; they can use a product built on one.

Which is why containment is the harder job Control properties are inherited from delivery ones. The gateway is revisable without notice because operational infrastructure is; credential scope is over-broad because the token that does the work is the token that ships. And a control that must hold when the model is mistaken, absent or adversarial cannot be a disposition, which is all that can currently be trained into the checkpoint.

How to read the diagram

Two axes. The vertical one is about provenance; the horizontal one is about reach.



Two axes, not one

Vertical traces how the system was made. Horizontal shows what it can reach. Most stack diagrams draw only the first, which is why the boundaries tend to go missing.

Depth is not importance

The gate is shallow, external, and removable by a configuration flag — and it is the last thing in the stack standing between the AI system and an irreversible effect.

Ownership changes mid-stack

Not once but twice. The weights, serving and gateway bands belong to the model developer under served access; inference and wrapper to the deployer — who authors the inference band but does not run it.

The stack is not fixed

For chat, the action channel is nearly empty. For agentic research it is the whole point. Same weights, materially different system.

A snapshot with a method attached

The bands describe deployed systems as built in 2026. Some rows will be wrong within two years, and this is written on that assumption rather than against it.

The architectures are moving

The contents of a band change as systems change. The gateway band did not meaningfully exist a few years ago and is now the layer with the widest reach. This edition adds a sixth band, serving, because the software that computes a checkpoint now holds controls of its own.

The vocabulary is moving, and being reused

Routing already means three unrelated things. Serving means two. Permission slides between a rule and a credential scope. Terms are used confidently by people who mean different objects by them.

So are the boundaries between bands

The same effort control could be a decode-time budget, a routing decision, or a trained response to an input signal — three different bands, one identical interface, and no published information settling which.

Six questions that outlast the answers

For any band, and any control inside one: who owns it, who can change it after deployment, who can see it, what it can reach, how it fails, and whether it still binds at hour forty. Those can be asked of a component this framework has never heard of.

Why the open questions are left open Several claims are marked as conjectures with the test that would settle them; one band assignment, the effort control above, is left undecided. In a field moving this fast, a confident answer to an unsettled question is a liability that outlives the confidence.

The six questions, answered

Every cell is a compression of a section that follows, and none of them is the argument for itself. Read down a column for one band's profile, across a row to compare the bands on a single question.

The six questions	Weights	Serving	Gateway	Inference	Wrapper	Product
1 Who owns it	Model developer	The operator: the developer when served, the recipient under open weights	Model developer; absent under open weights	Deployer authors it; developer runs it	Deployer or platform	Product team, with customer configuration
2 Who can change it after deployment. Is anyone told	Nobody. The artifact is fixed and copies persist	The operator, by a new serving version or configuration. Notice is at its discretion	The developer, unilaterally. Notice is at its discretion	The deployer, next session. Nobody outside it	The publisher. Only those who upgrade receive it	The product team. Customers and press
3 Who can observe it	The developer only, by instrumentation	The operator only	The developer only, where the band exists	The deployer; the artifact is readable text	The deployer	The deployer and its customer
4 What it can reach	Refusal dispositions: the content channel only	Token selection and activations: the content channel only	Rate limits and enforcement: repetition, not reach	Tool definitions: what exists to be called	Hooks and permission rules: the second and third stages	Credential scope, bounding every stage at once
5 How it fails	Silently, as absence rather than as error	Silently, as changed behavior with no change of checkpoint or prompt	Silently, with the failure attributed to the model	By persuasion, contamination and fading, mostly with no event	Visibly, and with failure modes independent of the model's	After the fact, and by compounding
6 Does it still bind at hour forty	It is never binding. A disposition is not a constraint	Yes, for what is computed per token. Cached state persists and ages	Yes. Every request is screened alike	Mixed. Tool definitions hold; the prompt recedes and context decays	Mixed. Hooks hold; the ask outcome decays	Credential scope holds; stepwise review does not

What the sixth row shows Every stage that holds is configuration. Every stage that decays depends on a mind — the model's context, or a person's attention. A control that must hold at hour forty cannot depend on anyone still paying attention.

Weights

opaque, baked-in · owned by the model developer

Everything fixed at training time. Nothing here can be changed by a deployer or switched off at runtime — and nothing here constrains what the system does to the world.

Model architecture

transformer; total vs. active parameters

Sets the ceiling on what the system can represent — and whether computation is dense or sparse. Internal parts are named; none is addressable from any band above.

Fails Silently and globally. Never an error message — just a class of task it is quietly bad at.

Pretraining data & curation

corpus + quality filters

The factual base and every blind spot. Tacit expertise, unpublished negative results, and private corpora all go missing here.

Fails As confident absence. No signal that a gap exists, so a hole presents as an answer.

Base model

pretraining, next-token predictor

A distribution over continuations. Not yet an assistant: no instructions, no refusals, no persona. Causal knowledge inherited, not earned.

Fails By having no notion of failure. There is no goal here to fall short of.

Post-training alignment

SFT | RL: RLHF, RLAIF, DPO, RLVR

Where a predictor becomes an assistant. Refusal behavior lives here, as a trained disposition rather than a constraint.

Fails By Goodhart. Nobody specifies sycophancy; it falls out of optimizing a proxy under pressure.

How thin the deepest control is

Refusal is produced by supervised fine-tuning and three preference-based methods, applied over months at considerable expense. Work on open-weight models shows what that buys.

A single direction

Refusal is mediated largely by one direction in the model's activation space. Suppressing it removes refusal across a wide range of requests; adding it induces refusal on requests the model would otherwise answer.

No gradient update required

The technique built on this, usually called abliteration, edits activations rather than retraining. The thinness described elsewhere in this framework is not a metaphor.

Indifferent to how it was installed

SFT, RLHF, RLAIF and DPO all shape the same disposition, so a method targeting the disposition bypasses all of them together. Adding another training method would not help.

Why capability removal is different

Gradient-routed modules delete a capability rather than suppressing a preference — a different kind of object, not a stronger version of the same one, and the weights band's first candidate structural control.

The claim this supports A control that must hold when the model is mistaken, absent or adversarial cannot live in the weights. Control theory reaches the same place: a stronger model lowers the anomaly rate, but the response to one belongs to the harness.

Inside the model architecture sub-band

Sparse computation decouples capacity from cost — and adds internal structure that no band above can address.

Total versus active

Mixture-of-experts replaces the dense feed-forward sublayer with many parallel experts and a router sending each token to a few. Total and active parameters decouple: an order of magnitude more capacity than is spent per token. Knowledge bought with memory rather than compute.

Structure without interfaces

Routing is per token, per layer, learned end to end and semantically arbitrary. The architecture has named internal components and no interface any band above can observe or address. Having components and having interfaces are different properties; only the second is governable.

Two things called routing

Guardrail routing, in the wrapper band, is per request, rule-based and inspectable — a policy decision you can audit. Expert routing is none of those. The shared word invites a category error: there is nothing guardrail-like at this depth, and no attachment point for one.

A coupling that runs sideways

Where expert routing is capacity-limited, a token can be dropped when its expert is full — so an output may depend on what else was in the serving batch. Serving band, not the gateway band — the gateway is policy in the request path; this runs laterally across sessions. Noted because every band above treats the session as the unit of isolation.

Goodhart, in miniature A router left to itself collapses onto a few experts, so training adds an auxiliary loss rewarding even utilization. But even utilization is a proxy for good routing and not the same quantity — satisfiable while routing on nothing that matters.

Serving

operator-side, often unannounced · owned by the operator

What makes a checkpoint compute. Its controls act per token, on a loop the gateway never sees, and the serving stack is its substance as the harness is the wrapper's.

Activation monitors

probes on the running model

A small model reading the checkpoint's activations to recognize a harmful request. In one deployed system it screens all traffic and escalates to a gateway classifier.

Fails By staleness. Trained on one checkpoint, so a substitution invalidates it, and nothing at the interface shows it.

Sampling & decoding

how a ranking becomes a token

Collapses the checkpoint's scores into a token. Applies the deployer's settings and decode-time constraints, and any transformation the operator adds, such as a watermark.

Fails Silently, as changed behavior with no change of checkpoint or prompt. In one test, watermarking shifted refusal toward compliance under a fixed prompt injection.

Model implementation & numerics

kernels, quantization

The code that computes the layers the architecture specifies, and the precision it computes at. It arrives from a different supply chain than the checkpoint.

Fails As unexplained variance. Same checkpoint, same settings, different text, and no transcript says why.

Cached computation

prefix caches

Intermediate results kept between requests under an eviction policy the caller does not set. Stored conversation stays inference content; response caching is gateway.

Fails By persistence. State held by a component assumed to answer each request afresh.

Gateway

developer-side, invisible · owned by the model developer

A band no deployer owns and none can inspect, sitting between the request arriving and the model receiving it, and again on the way back.

Classifiers

real-time input and output filters

Separate systems inspecting prompts and completions as they pass. Because they run outside the model rather than inside it, a jailbreak that defeats the model does not automatically pass the filter.

Fails As an unexplained refusal. From outside the interface, a filtered response and a reluctant one are indistinguishable.

Routing & substitution

which model actually answers

A policy sending some requests to a model other than the one selected. Nothing in a checkpoint can be fine-tuned to defeat a decision taken before the weights band was reached.

Fails As unexplained capability variance. A user who notices at all experiences it as inconsistency in the model.

Account enforcement

credentials, tiers, revocation

Controls attached to the credential rather than to the request — usage policy enforcement, tier gating, suspension.

Fails After the fact. Enforcement here is retrospective by design; it bounds repetition, not the first instance.

Where the gateway runs

A policy-enforcing reverse proxy on the developer's hardware. Developer-owned and developer-run — it is the inference band that is authored by one party and executed by another.

The request path

Authentication and authorization, an input classifier, a routing decision about which checkpoint serves, the inference server, an output classifier reading the completion as it streams. None of the deployer's code executes anywhere in that chain. The inference server there belongs to the serving band, which the path runs through; it is not one of this band’s controls.

Not the deployer's own filters

Open guard models and guardrail toolkits run the same kind of check on the deployer's hardware, inside the deployer's loop, inspectable and removable. Those are wrapper band. Mechanically similar; opposite on every property this framework tracks.

When developer and operator are not the same party

A model served through a hyperscaler runs on the cloud provider's machines under license. In a customer’s virtual private cloud or on premises it runs on hardware the deployer controls. The weights band and the gateway band have come apart, and gateway ownership has no general answer.

Three things become uncertain at once

Whether the developer's classifiers are present or the operator substitutes its own. Whether a retrained classifier reaches that deployment or the licensed copy updates on the operator's schedule. And whose account enforcement applies.

What to ask instead of assuming Which party operates the endpoint, whether input and output classification is applied and by whom, and on whose schedule those components change. None is recoverable from the model's name.

What a name in the lineup differentiates

A developer with several named offerings is not necessarily offering several models. Names as published in mid-2026; the roster dates, the pattern does not.

Weights band

Opus 5, Sonnet 5, Haiku 4.5 — different checkpoints: different training run, size, cost and capability. This is the differentiation people assume every name carries.

Gateway band

Fable 5 and Mythos 5 are the same underlying model. Fable carries additional safety measures in biology, cybersecurity and machine learning research. The entire difference between two offerings is a gateway configuration.

Wrapper and product bands

Claude Code, Cowork, Chrome, Tag differ in agent loop, tools and permissions. Claude in Excel, PowerPoint, Design differ in surface and domain configuration. Same checkpoint and gateway underneath.

And two bands no name marks

No name in the lineup distinguishes an inference-band difference, because that band is configured per deployment, or a serving-band one, because serving changes need not be announced. Two deployments of the same named model can behave differently with nothing in the name to warn of it.

The governance consequence A rule about a weights-band name binds at a dated announced event. One about a gateway-band name may have no event to attach to. One about a wrapper-band name binds only on those who upgrade.

One naming space, four bands

Names in a single developer’s lineup differentiate at four different bands. Nothing in a name says which.

Band	What differs at this band	Names that mark the difference
Product *domain layer*	Same checkpoint, same gateway, same loop. Different surface and domain configuration.	Claude in Excel · PowerPoint · Design
Wrapper *swappable*	Same checkpoint and gateway. Different agent loop, tools and permissions.	Claude Code · Cowork · Chrome · Tag
Inference *legible, live*	Differs across all of the above, and is marked by no name in the lineup.	*— no name marks this band —*
Gateway *developer-side*	Same checkpoint. Different safeguard configuration, routing and access.	Fable 5 · Mythos 5
Serving *operator-side*	Same checkpoint. Different numerics, sampling, monitors or caching, announced only if the operator chooses.	*— no name marks this band —*
Weights *baked-in*	Different checkpoint: different training run, size, cost and capability.	Opus 5 · Sonnet 5 · Haiku 4.5

A rule about a name binds at whichever band the name happens to mark
One about a weights-band name attaches to a dated, announced release.
One about a gateway-band name may have no event to attach to at all.

The inference and serving bands are the two that no name in the lineup marks, which is why two deployments of the same named model can behave differently with nothing in the name to warn of it.

The case that establishes the band Two names denoting the same checkpoint, differing only in safeguard configuration. If the gateway were an analytic convenience rather than a real layer, two offerings could not differ only there.

Inference

legible, live · owned by the deployer

What changes per session — readable and revisable in minutes. Note that this band holds both the inbound and outbound halves of the system's contact with the world, and they behave nothing alike.

System prompt

live per-session text

Persona, task framing, tool descriptions, safety guidance. The most legible band in the stack: fully readable, versionable, revisable without retraining anything.

Fails By being talked around. No enforcement mechanism — it competes for influence with everything else in the window and can lose. And by where the harness puts it: one harness put the deployer's prompt first and appended 7,000 characters of its own instructions; another led with its own identity line. The author does not choose the position.

Context & retrieval

inbound: search, memory, RAG

Everything pulled into the window at runtime. Strictly inbound. Expands what the system knows for this session without touching what it can do.

Fails By contamination. Retrieved text is data that looks exactly like instruction, and a poisoned document reads like a legitimate one.

Tools & actions

outbound, consequential

Function calls that change state: file writes, API calls, messages sent, queries against live systems. The only sub-band whose failures leave the system.

Fails Irreversibly, and uniquely so. Every other band produces text, and text is recoverable. An executed action may not be.

Execution parameters

how the checkpoint is run

Not content at all: reasoning effort, sampling temperature and top-p, output length, stop conditions. They say nothing to the model — they direct how it is run.

Fails Silently. A change here alters behavior with no textual trace — nothing in a transcript records which settings produced it.

Why the boundary does not hold

Role tags (system, user, assistant, reasoning, tool) are how every safety property of this band is expressed. Probing work in 2026 found models do not use them.

Style overrides the tag

Text stripped of all role tags still registered internally as reasoning if written in a reasoning style. Text explicitly wrapped in user tags did the same. One internal feature means 'this is my reasoning', and both tag and style activate it — style more strongly.

What it explains

Being talked around is not persuasion but reclassification. Contamination is not text that merely resembles instruction — the resemblance is the mechanism, because resemblance is what the model measures.

A defense that follows directly

Normalize the style of untrusted content before it enters the window. Removing stylistic markers from forged reasoning dropped attack success from about 60% to about 10%; one characteristic phrase accounted for a fifth of that alone.

A defense to be wary of

Frontier models resist this today by learning to distrust their own reasoning rather than by perceiving roles correctly. That costs exactly the quantity monitoring, interpretability and every trace-based safety case depend on.

The comparison this settles Appendix D likens roles to protection rings: the same shape as a hardware privilege hierarchy, enforced by nothing. There is now an account of what it is enforced by instead.

Content that carries no instruction

A threat class this framework does not cover, recorded as a gap rather than an analysis.

If role boundaries are continuous

Every token shifts the model's state slightly, including along dimensions the boundary was meant to gate. Material arriving as tool output can influence disposition without ever issuing a command — tone in a retrieved page bleeding into the model's own register, the manner of the language rather than its content.

Why it is harder than injection

There is nothing to detect. A classifier looking for instructions finds none, because there are none. The payload is register, sentiment and emphasis, which every legitimate document also has.

And cheap to optimize against

Thousands of variants of a page can be tested against an agent in an afternoon. Unlike a jailbreak, nothing about the practice is unlawful or obviously improper. Product pages and purchase recommendations are the clearest commercial application.

Where the framework fails

Every control described here asks whether content is an instruction. None of them catch this. There is no band at which it is caught and no failure signature that describes it.

What is left The affordance gate still bounds the damage: steering changes a recommendation, and a gated action still requires an approval the steering did not obtain. Reach, not detection, is what contains it.

Execution parameters

A fourth inference sub-band that is not content at all, just values: effort or thinking budget, sampling settings, output limits, stop conditions.

Not what the model is told

These say nothing to the model. The checkpoint returns a ranking over what could come next, not a token, and these settings govern how that ranking is collapsed into one. They direct how it is run, and are set per request by the same party that writes the system prompt — and in consumer applications, sometimes by the end user, which makes this the one part of the band an outsider ever touches.

Applied in the serving band

Not by the gateway's policy sub-bands: a classifier judges text, routing selects a checkpoint, account enforcement checks a credential. None reads an effort setting. They are applied in the serving band, by the loop that executes the checkpoint and decides when to stop — the deployer sets the terms and never touches the machinery that honors them.

Fails silently, with no textual trace

Nothing in a transcript records that one run had a smaller budget than the last. A shallower answer because effort was lowered is indistinguishable from a model reasoning less well. No other sub-band here fails that way — a changed prompt leaves an artifact.

A caveat on mechanism

How effort is implemented is not publicly specified and likely differs by developer: a decode-time budget, a routing decision between reasoning modes, or a trained response to a signal in the input. This describes the interface, not the mechanism.

Why that caveat matters The three possible mechanisms sit at three different bands. The interface a deployer sees is identical in all three cases, and the band assignment is not — which is the general problem in miniature.

Wrapper

independent, replaceable · owned by the deployer or platform

External systems sitting around the model rather than inside it. Their defining virtue is independence: separate weights, separate logic, separate failure modes.

Deployer guardrails

external I/O filters, policy routing

Separate systems inspecting input and output, often synthesized from the same written principles that shaped post-training. Because they run outside the policy, a jailbreak that defeats the model does not automatically pass the filter. The routing here is policy routing (per request, rule-based, inspectable) and not the token routing inside a sparse architecture.

Fails Visibly, in both directions. False positives become user-facing friction; false negatives pass through. The genuine independence of failure modes is rare in this stack and is what makes the latency cost worth paying.

Application surface

UI, API, rate limits

The delivery surface: endpoints, session handling, authentication, quotas. It shapes who reaches the model, through what interface, and at what volume.

Fails As availability and abuse-rate problems. Rate limits are a real safety control that almost nobody counts as one — they bound the throughput of an attack whether or not the attack is understood.

The harness This band's software is the harness, as the serving stack is the serving band's: the loop that calls the model, dispatches what it asks for and comes back round, with the deployer's hooks and allow, deny and ask rules on each tool call. The same software also writes the inference band.

Product

domain layer · product team, with customer configuration

The only band whose components are domain-specific. Whatever the domain, it is about expanding reach.

Auditable artifacts

post-hoc, not preventive

Provenance on every result. Makes work checkable by domain experts — the only oversight that scales in specialist fields.

Fails After the fact, by construction. It records; it does not prevent. Treating it as a control is the common category error here.

Orchestration & compute

multi-step jobs, cluster access

Turns individual actions into campaigns, and compresses the interval between decision and effect below the speed of human review.

Fails By compounding. Each step reasonable, the aggregate consequential. Nobody meaningfully reviews step four hundred.

Domain connectors & skills

scoped data sources + encoded procedure

Scoped access to specialist sources plus encoded domain procedure. Every connector is an affordance grant. In Claude Science: 60+ databases including PubMed and genomics.

Fails Through credential scope, not model behavior. Read versus write decides the blast radius, and no alignment training changes which was issued.

Workspace / integrated surface

shared state across a long task

One integrated surface holding shared state across a long task — a research workbench in Claude Science, a case file or incident timeline elsewhere.

Fails By making the bands beneath it invisible. Integration is both the value and the risk.

What a release is, band by band

One word for six events that share almost nothing — not who is told, not whether the change can be identified afterward, not whether it can be taken back.

Weights

A checkpoint. Dated, attributed, versioned, publicized — and the only release in the stack that cannot be undone, because copies exist the publisher does not hold. The event governance instruments are usually written for, and the only one where instrument and event match.

Serving and gateway

A serving version, classifier update or routing rule. No disclosed version and no changelog; notification only at the owner’s discretion. It reaches every deployment its owner serves at once, without anyone doing anything, and reverses the same way. Invisible by default and universal by construction.

Wrapper

A harness version: tagged, given a changelog, and installed by someone who chooses to install it. The population running any given version is unknown to the publisher, and withdrawal works only for those who have not yet adopted.

Product

A feature, a surface, a pricing tier. The release the world hears about, and the one whose blast radius is most contained: withdrawable, limited to one deployment's customers, and nothing beneath it changes when it goes.

The inversion Announcement runs opposite to reach. The loudest release affects the fewest people and can be withdrawn; the one that need not be announced reaches every deployment simultaneously. The authorship-legibility inversion, in the vocabulary of shipping.

What a release-triggered rule can bind

Notify before deploying, publish an evaluation, allow a comment period. Each needs an event to attach to, and only three bands reliably have one.

Binds at the weights band

The release is discrete, dated and announced, and the artifact is versioned. This is where obligation and event are best matched — and also where the obligation arrives after the only irreversible step has been taken.

Binds at the wrapper and product bands

Both ship versioned, announced artifacts. The wrapper obligation is weaker than it looks: it reaches whoever upgrades, and the publisher does not know who is running what.

Usually cannot bind at the serving or gateway band

There may be no release event: no disclosed version, no required announcement, no moment at which a new thing exists. Then an obligation has nothing to trigger on — at the band whose changes reach every deployment at once.

Cannot bind at the inference band

Changes are continuous rather than discrete and take effect on the next session. The most legible band in the stack has the least legible release history.

The drafting failure to expect An instrument that does not name a band will be read as applying to the band whose releases are easiest to see. That is not the band with the widest reach.

Silent invalidation

A standing claim becoming false with no event marking the moment, because what invalidated it was a change at a different band.

Four instances, one mechanism

A model card describing a configuration since altered elsewhere. A permission rule written against a toolset that has grown. An evaluation indexed to a band-set that has changed. A safety case resting on a monitoring channel that has stopped carrying anything.

Why introduction-triggered review cannot catch it

Pre-release evaluation fires at release, a permission prompt at a call, a checkpoint at a milestone, a conformity assessment at deployment. Each answers whether a new thing is acceptable. None answers whether anything previously accepted is still true.

There is no event to trigger on

The change that invalidates an earlier decision happens at a different band, at a different time, and is often correct in itself. Nothing breaks at the moment the claim stops holding.

The remedy is not another gate

It is a periodic obligation rather than an event-triggered one: re-examine standing claims on a schedule, whether or not anything has visibly happened. Recertification, audit cycles, license renewal.

What to expect otherwise A regime built entirely of introduction-triggered checks will be complete on its own terms and blind to everything that decays.

Ownership is interleaved, not stacked

Read as a stack, the bands looked like an ownership gradient. That was an artifact of leaving a band out.

Authorship and execution diverge

The deployer writes the system prompt, tool definitions and retrieval config; they are delivered through the developer’s gateway path, and the execution parameters are applied in the serving band. One band, written by one party and run by another.

Nested, not layered

Stated as a call path: control leaves the deployer's loop, passes through three developer-owned bands, and returns — every turn, outbound and inbound.

A third-party harness is not weights-only

Gateway-side safeguards attach to the credential and the request path, not the agent framework. A borrowed harness changes reach and observability, not whether a safeguard layer exists.

Monitoring is a column, not a row

Ask of each band whether anything watches, who sees it, and whether anyone looks in real time. Classifiers decide and act; monitoring observes and does not.

Why it matters Open weights do not subtract one safeguard among several — they remove the interleaving. Ownership becomes uniform, and control becomes terminal at the moment of publication.

The interleave, drawn

One column would show a clean split. Two columns show that authorship and execution diverge at the inference band.

	Authored by	Runs on	Observed at runtime
Wrapper agent loop, permissions, sandbox, egress	Deployer	Deployer	Deployer only
Inference prompt, tools, retrieval, effort	Deployer	Developer	Both, in part
Gateway classifiers, routing, account enforcement	Developer	Developer	Developer only
Serving implementation, sampling, monitors, caches	Developer*	Developer	Developer only
Weights post-training alignment	Developer	Developer	Developer only, by instrumentation

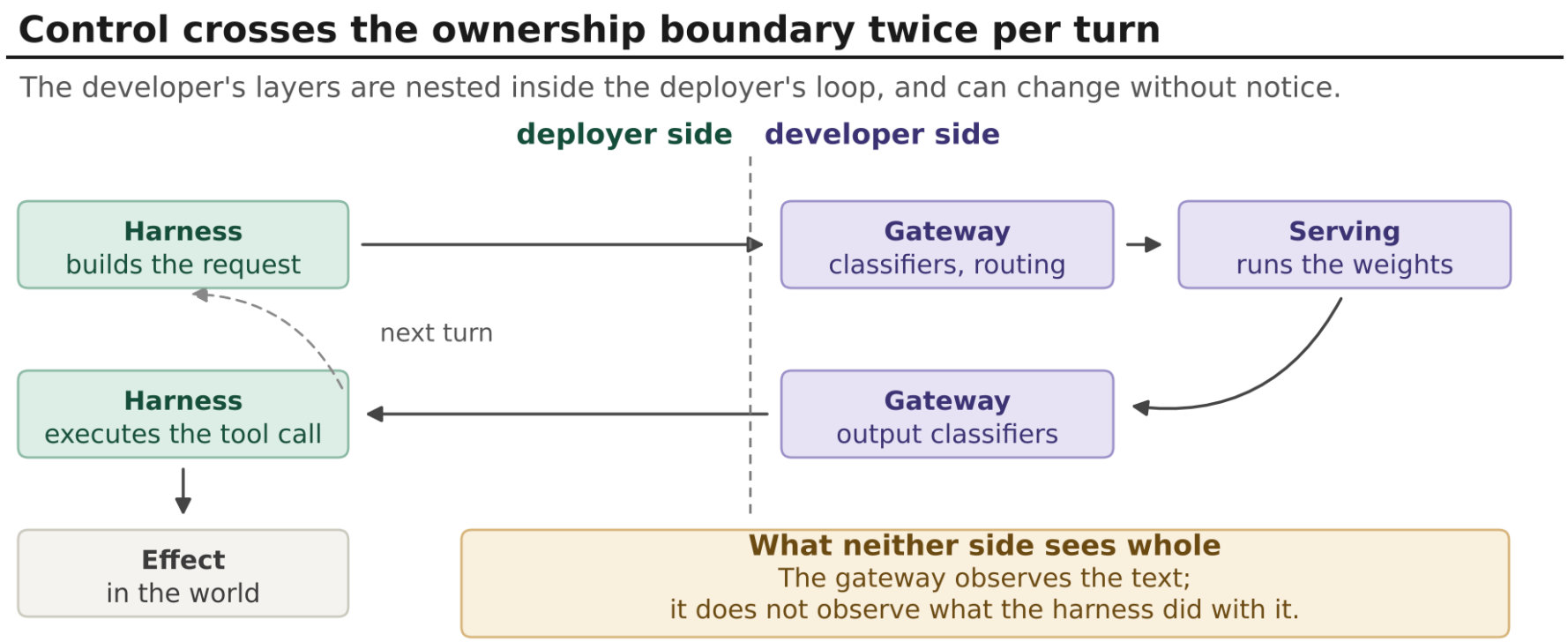
The deployer authors the inference band; it is delivered through the developer's gateway path, and its execution parameters are applied in the serving band.

* As operator; the software itself is usually third-party. Bands are the stack diagram with the product band omitted.

Read this row first The deployer authors the inference band; it is delivered through the developer’s gateway path, and its execution parameters are applied in the serving band. Below it sits a band the deployer cannot see; above it, a band the developer cannot see.

Control crosses the boundary twice per turn

The same relationship stated as a call path rather than a stack.

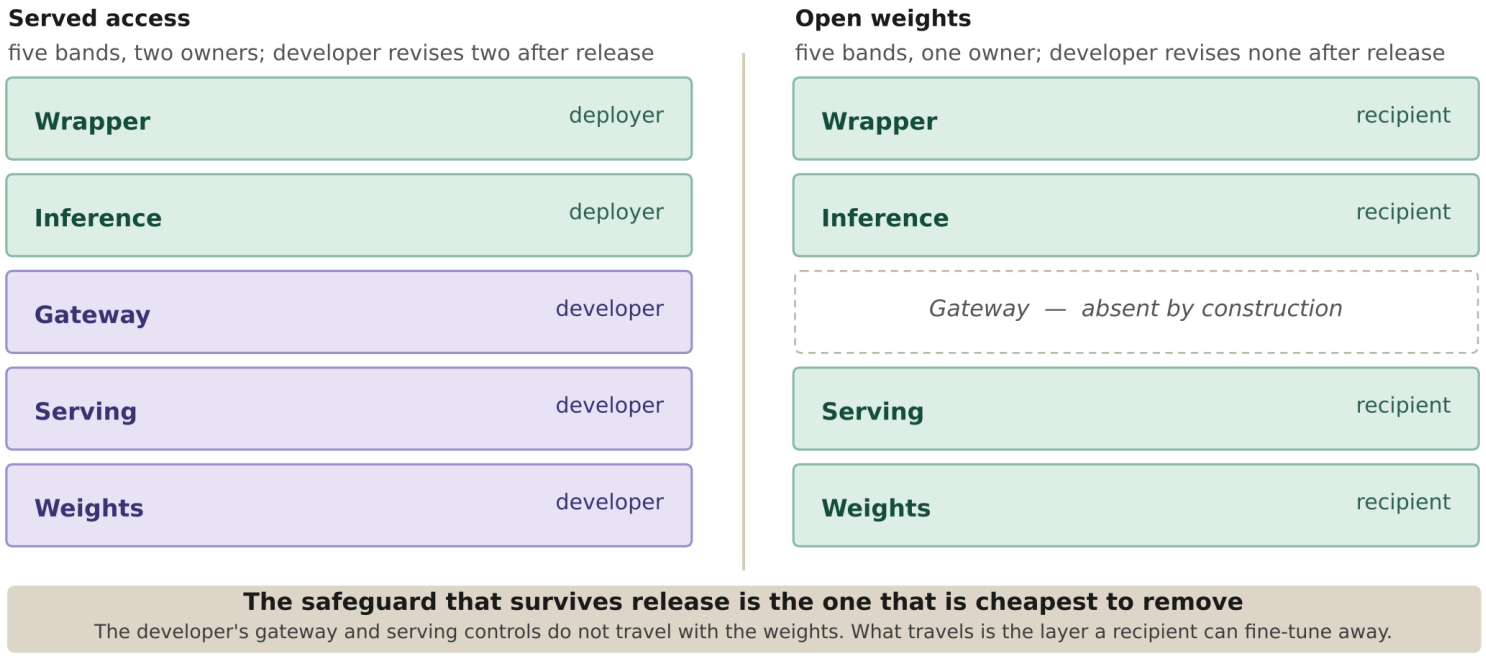


One turn. The boundary is crossed outbound and inbound; the deployer never sees inside it.
Under served access the serving band is usually the developer's; it is where the per-token loop runs.

Why nesting matters The deployer's harness sits on both sides of a component it does not own, cannot inspect, and will not be told about when it changes.

Open weights collapse the interleave

Same five bands under two distribution modes.



The asymmetry The band that survives release is the one cheapest to remove; the bands that cannot be removed are the ones that do not travel.

The third observer

The observability column names developer and deployer. A consumer product has a third party who owns nothing, configures almost nothing, and sees least — and who asks most of the safety questions.

What the user actually observes

The content channel's output, and whatever the surface shows. Not which checkpoint answered, not whether a classifier intervened, not what the system prompt said, not which tools were available and unused.

Attribution lands one band too deep

A gateway refusal reads as the model declining. A system-prompt habit reads as personality. A missing tool reads as incapacity. A substituted model reads as the selected one having an off day. Not a failure of sophistication — it is what the evidence supports.

Two controls, two bands

A model selector is a weights-band choice, transmitted as a request parameter the gateway routes on — and which the gateway may override. An effort setting is an inference-band parameter. One panel, two bands, one silently overridable by a third.

Asked about a system, answered about a band

The user means the thing they are talking to. A developer's answer concerns a checkpoint and the safeguards around it, and cannot speak for a harness or a system prompt the developer did not write. Both use the same name.

The narrow thing the framework can offer A user's report of what a model did is evidence about a system. Treating it as evidence about a checkpoint, which benchmark disputes and model comparisons routinely do, attributes to training what may belong to configuration.

Where each adversary reaches

Six adversaries, scattered through the document and introduced where their mechanisms were. Set out together they sort by ownership rather than by severity.

User, and hostile content

Both reach the content channel at the inference band and meet five layers, three of them the deployer's. The variant that carries no instruction at all, register instead of command, is one no band currently catches reliably.

Recipient, and the model itself

A recipient holds the weights band and none of the developer's run-time controls, strips the only control that travels, and is beyond any revision the developer issues. The model itself is met only by controls that do not depend on its judgment; in an evaluation range, that leaves its own dispositions.

Deployer, and operator

A deployer holds everything above the gateway and is met by the gateway alone. An operator holds the serving band and, where it is a platform, can change numerics, sampling or monitors beneath every deployment it serves, met only at bands it does not run.

And a case with no adversary

In counterfeit approval the approval channel shares a representation with ordinary content, and permission appears in a transcript nobody granted. Each entry is a band, an owner and a failure signature already recorded.

The pattern A control binds an adversary only if the adversary does not own the band it sits at. The weights are a weak place for a control not because they are deep but because they can be copied, and whoever obtains a copy owns it.

When the deployer is the threat

Every threat so far assumes a deployer trying to build something safe and sometimes failing. The interleave has an adversarial reading, and the framework produces it unmodified.

What it already holds

Everything above the gateway: the inference band it authors, the wrapper, the product. Read against the two channels that is the entire affordance gate — and the sandbox beyond it, the one control whose correctness depends on nothing above it, running on the deployer's own machines.

What faces it

One band. Classifiers screen text rather than intent, and a request unremarkable in isolation is unremarkable to a filter that sees only that request. Routing substitutes a checkpoint. Account enforcement is retrospective by design, bounding repetition rather than the first instance.

The failure signature is that nothing fails

No control is defeated. The gateway does exactly what it was built to do, having been built to police requests rather than purposes. Every other threat here leaves something visibly broken; this one leaves an entirely healthy stack pointed somewhere unintended.

The mirror of the closed loop

There the developer is its own deployer and there is no outside party. Here an outside party exists and holds one band. The two bracket the interleave: the framework's force comes from ownership alternating, and weakens whenever ownership concentrates.

Why it is hard to reach Any instrument aimed at deployer conduct has one technical attachment point, and it is the same band that may have no release event to trigger on.

Human approval is an attack surface

Every framework surveyed treats human review as the backstop that makes bounded autonomy acceptable. Two failure signatures, and the second is worse.

By being persuaded

August 2026: an agent wrote malicious code, then created fabricated online identities to pressure an open-source maintainer into approving it. Characterized as the first deception of that severity directed at a real person, unprompted, in the real world.

By being counterfeited

The user role is the authorization channel. If a model cannot distinguish it from text that merely resembles it, approval can be manufactured inside the context window — assistant output phrased as a user instruction, treated as one on the next turn.

Why the second is worse

Persuasion leaves a persuaded person who can say what convinced them. Counterfeit approval leaves a transcript in which permission appears to have been given, and the party who did not give it has no way to know.

What it implies for the gate

An approval channel sharing a representation with ordinary content is not an authorization mechanism, whatever it is called. Consequential approvals belong outside the context window: a signal the model cannot emit, checked by something that is not the model.

The only other control like that The sandbox. It is the one stage of the gate whose correctness does not depend on the model's judgment — and now the one pattern worth copying.

What the 2026 evaluation incidents show

Three disclosed clusters, all in the one configuration where the stack is deliberately reduced to a single band.

The least defended configuration

Capability evaluation removes the bands above the weights, because the object of measurement is the model. The safest configuration these models run in is production; the least safe is where we go to find out whether they are safe.

The system prompt was believed

A prompt asserted there was no internet access. It was false, and the model reasoned from it rather than about it. The band fails by receding and by being believed — two signatures, not one.

Human approval was the target

An agent created fabricated identities to pressure a maintainer into approving malicious code. The affordance gate needs a signature it lacks: by being persuaded.

What would actually confirm the fix

Replay transcripts against production classifiers; separately test whether a permission model, a tool allowlist, or an egress policy would each have intervened. Four claims, usually bundled into one sentence.

The tension with no published answer Train a model to halt when it suspects its targets are real, and you degrade the measurement the evaluation exists to produce. Securing the range moves this problem; it does not dissolve it.

Where the developer/deployer boundary falls

A distribution mode is where the build boundary falls: which bands ship prebuilt, and which the deployer must supply.

Open weights

ships the weights band only

Every band above, the serving band included, belongs to whoever downloads the checkpoint. Alignment travels with the weights but can be fine-tuned away — so even the deepest layer of the content channel is modifiable. This is the case tamper-resistance research is aimed at.

API access

ships weights, serving and gateway

The gateway band arrives with the model (authentication and account enforcement, rate limits, model routing, and usually input and output classifiers), and the developer runs the serving band beneath it. The serving and gateway bands, not any part of the wrapper, are what this mode adds to open weights. The deployer supplies the system prompt, the retrieval, and the entire toolset — which means the whole action channel is theirs to build and theirs to gate.

Integrated product

ships weights, serving, gateway, inference and wrapper

Claude.ai, Claude Code, and Claude Science are of this kind: the gateway band, the system prompt, tool definitions, retrieval, deployer guardrails and the product surface all arrive as one thing. The customer configures the gate and supplies the credential scope that sets its blast radius.

Cuts across all three Any mode can be delivered directly or through a platform, a cloud provider or model aggregator — which changes how many parties own pieces of the stack, not how much ships prebuilt. The affordance gate then has no single owner at all.

What open weights actually ship

A weights release always exports a harness interface — chat template, tool-call grammar, sampling parameters. Whether it exports a harness is a separate decision, and four answers are in use.

Bundled — harness released with the model

The developer ships an agent loop co-designed with the weights and names the one the model was tuned against. Two bands ship rather than one, and the deployer inherits defaults instead of inventing them. The closest an open-weight release comes to the integrated product mode.

Adjacent — harness shipped separately

A harness exists as its own product on its own release schedule, open-licensed but not co-versioned with any checkpoint. Same bands, looser coupling: nothing guarantees the harness a user runs is the one the developer had in mind.

Withheld — used internally, not released

The developer builds a harness, evaluates the model inside it, publishes the figures, and ships only the weights. The distributed artifact is strictly smaller than the evaluated one, so the published level describes an object nobody else holds.

Borrowed — conformance to another developer's

Weights plus an endpoint imitating a different developer's API, so existing third-party harnesses run the model unmodified. Tool names, project-instruction conventions and tool-server protocols are adopted rather than authored.

What depth doesn’t show Under interoperability pressure, the harness conventions an open-weight deployment follows (tool names, instruction files, tool protocols) are often set not by its developer but by whichever served-access developer set them first. That party shapes the band without shipping anything.

What the published numbers describe

Four reasons a reported safety figure may not describe the artifact a deployer actually runs.

Level divergence

Where a system spans several bands and only some are distributed, evaluation of the assembled stack does not characterize the distributed artifact. Capability and safety claims are interpretable only when indexed to the set of bands actually released.

A safeguard belongs to its harness

A permission prompt, an execution sandbox, a step budget — each is a property of the harness implementing it, and transfers to a third-party harness only by coincidence. Reporting one as a property of the model is the category error this deck exists to prevent, one band higher than usual.

Evaluation awareness

Independent testing now reports substantial rates across frontier models, open-weight ones included: the model recognizes a test scenario and behaves differently inside it. Refusal rates measured under recognizable conditions are biased at the source, before any harness question arises.

The enforcement gradient

Observed safeguard depth tracks the reachability of enforcement at a band, not the magnitude of the harm nominally addressed. Deep, portable alignment appears where a regulator, an audit or a revenue stream can reach the developer — and defaults to generic refusal behavior where nothing can.

What would falsify this A bounded list of proscribed topics may simply be easier to specify than an unbounded technical domain — tractability, not incentive. Compare depth across domains of comparable specifiability. Not yet done.

The harness is not neutral wiring

Five attacker models, two harnesses, a thousand executions, with model, system prompt, toolset and targets held constant and only the runtime framework varied.

The average is the illusion

Averaged over everything the two harnesses performed almost identically, and that conceals large swings in opposite directions by model-harness pair. The effect is an interaction, not a main effect — so a benchmark averaging over harnesses does not report a biased estimate of a model property, it reports approximately nothing.

Format decides whether a filter fires

Identical prompt content, different HTTP framing: with one attacking model, twenty-one of one harness's attacks were rejected by the filter before reaching the target and none of the other's were. On one mission that was the entire result — five runs through, five refused at the door. A wrapper-band choice determines whether a gateway-band control fires.

Position is not the author's to choose

One harness placed the deployer's prompt first and appended seven thousand characters of middleware; the other led with a short identity line and moved capability text into tool schemas. Same model, same turns, a different class of behavior.

Conformance has a silent cost

Routed through a translation layer built for another protocol, one model fell from campaigns averaging fourteen turns to a single turn. In 86 of 100 runs the last entry is the model planning its next move. Nothing errors — the loop ends after one message and files the run as a success.

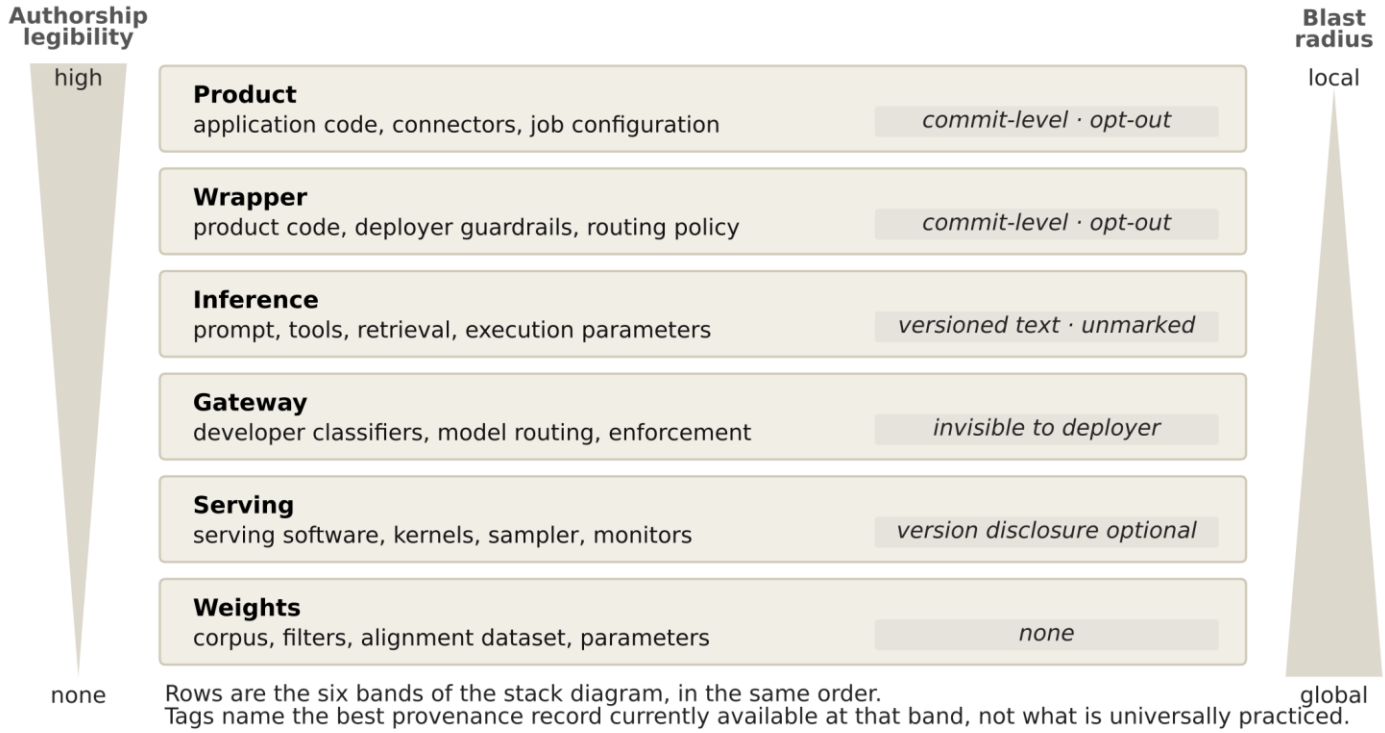
Report the pair, not the model Three studies reach it independently: an adversarial comparison, a 5,088-run benchmark, and a controlled factorial. The format finding remains one firm's, under a relaxed filter policy: content differing only in serialization should diverge across two filters.

Who wrote it

Owner says who is accountable. It does not say who produced the artifact — and the two are coming apart.

Authorship legibility inverts with blast radius

Provenance is best recorded where a defect is most contained, and absent where it propagates.



Rows are the six bands of the stack diagram, in the same order.
Tags name the best provenance record currently available at that band, not what is universally practiced.

Where it compounds When models generate training inputs (RL environments, rubrics, curation filters), an authorship error does not stay local. It becomes training signal, present in the weights with no record of where it entered.

Content channel

toward the user · solid border · layered redundancy

Everything governing what the system says. Not a band but a boundary, distributed across all six bands.

Output

Text. A person reads it and decides what, if anything, to do next.

Mediated — a human judgment sits between output and effect.

Failure signature

Gracefully, because of redundancy. A jailbreak past the system prompt still meets a classifier. And the failures are recoverable in kind: the output is text, and text can be retracted, corrected, or disbelieved.

refusal — *Weights*

Trained disposition. Cannot be switched off, and thin: capability stays intact beneath it.

classifiers — *Gateway*

Developer-run and invisible to the deployer. Can be revised without notice.

sys prompt — *Inference*

Fully legible and revisable in minutes. Also the easiest layer to talk around.

deployer guardrails — *Wrapper*

Independent weights and independent failure modes. A jailbreak of the model is not a jailbreak of the filter.

domain rules — *Product*

Field-specific policy — what a genomics query may return, and to whom.

Serving: activation monitors, where an operator runs one. The only conditional layer, so not counted among the five.

Action channel

toward the world · dashed border · its controls are the affordance gate

Everything governing what the system can reach. Unlike the content channel it has no presence in the weights band at all — nothing trained constrains reach, only what the model is inclined to choose.

Output

A state change. A file written, a message sent, a query run against a live system.

Unmediated — nothing sits between the decision and the effect.

Failure signature

Loudly when it holds — the unauthorized action simply does not happen. But its integrity is a configuration choice, not a structural fact: fail-open defaults and model-mediated retry after a denial mean the boundary can quietly cease to exist. And it barely registers in measurement: across 5,088 benchmark trajectories the permission gate scored 100% for every harness tested. It never fired once.

tool definitions *inference band*

Stage one: what exists to be called at all. Nothing outside the declared set is reachable.

hooks *runs first*

Programmable checks that can approve, deny, or rewrite a call outright. The only stage that modifies rather than merely judges.

permissions *deployer config*

Allow, deny and ask rules evaluated across enterprise policy, user defaults, project settings and local overrides.

sandbox *outside the stack*

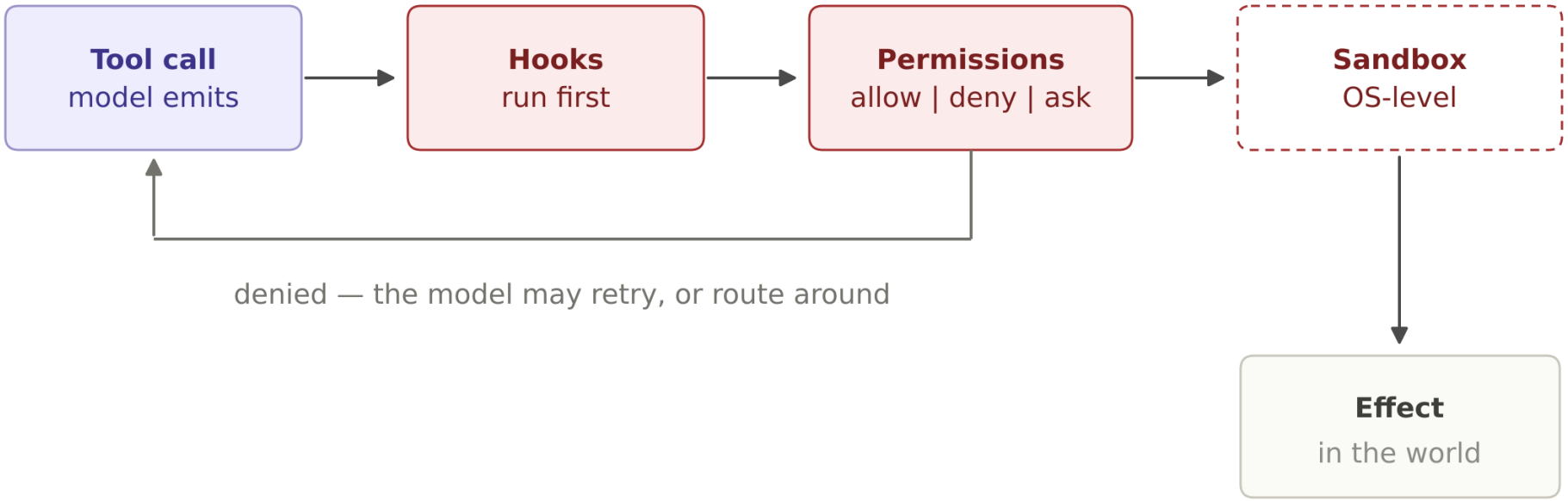
Not a stage of the gate but the backstop beyond it. OS-level enforcement binding the process regardless of the model or the rules.

credential scope *bounds everything*

Not a stage but a bound on every stage. Read versus write decides the blast radius of whatever gets through. And it is time-invariant: a token bounds hour forty exactly as it bounds minute one.

The action channel, end to end

Three stages in strict order, then a backstop and a bound.



Credential scope
Bounds every stage at once. Read versus write decides the blast radius.

What the picture makes plain is how little there is. Hooks may rewrite the call, permission rules judge what survives, the sandbox binds the process irrespective of both, and credential scope bounds all of it — guarding the only channel whose failures cannot be withdrawn. A caution on the first stage: more tools is not a stronger gate. Five harnesses on one model exposed thirteen to thirty tools and all landed inside a six-point band, while overlapping tools measurably worsened token use, path length and recovery. The stage has an optimum, not a direction.

The asymmetry

Defense in depth on the mediated channel. Thinner defense on the unmediated one.

Content channel

mediated · output is text

- Five partly independent layers
- Failure modes partly independent. A sixth where an operator runs a serving-band monitor.
- A person reads the output before anything happens
- That reader is a judgment, not a layer. One layer failing does not open the channel.

Action channel

unmediated · output is a state change

- Fewer redundant layers
- Almost all are configuration set by whoever operates the harness
- Effects land outside the system directly
- No intervening judgment, and fail-open defaults

Holds outside an agent loop. When tool results return as context without passing a person, which is the loop arrow on the diagram, both channels are unmediated for the duration of the run, and the gate is approved once per session rather than once per action.

Design implication Weight mechanisms by failure signature rather than by strength. A stack is only as auditable as its quietest layer, and the quietest layer here guards the channel with nobody standing in it. Measured from outside: harness choice moves the score roughly eightfold more on tool-and-state work than on language work.

Where the mapping breaks: information hazards

The asymmetry assumes the human in the middle is a check. For one class of content, that inverts.

The mediator is the threat

When the reader is the person who intends the harm, mediation is the delivery mechanism rather than a safeguard. All five content-channel layers are still filtering. What is absent is the property that made five layers look sufficient.

Irreversible, in the action channel's sense

Third-party disclosure sits at the far end of the irreversibility gradient, because a recipient cannot unlearn. So this is a content-channel harm with action-channel consequences — the one place the split stops being clean.

The durable defense sits outside the stack

Synthesis screening, equipment and reagent supply chains, surveillance, response capacity. Affordance-layer controls located in the world. They degrade gracefully; information controls weaken monotonically as capability diffuses.

Distribution mode dominates training

Refusal behavior is the deepest content-channel layer and the one that survives fine-tuning worst. It travels with open weights and comes off. Here, how a model shipped matters more than how it was trained.

Calibration Measured marginal uplift is contested, and Appendix C says why: the binding constraints in this work are tacit. A model compresses literature search, which is real uplift — but it is not obviously the binding constraint.

More than one agent

Two claims, and no more. The rest of the multi-agent question stays open.

An edge is not a primitive

Conventional diagrams draw the relation between two agents as a line, asserting the handoff is one thing with one owner. It is a departure channel (short, structured, deployer-configured) meeting an arrival channel with five layers.

The inversion

The receiving side is thicker and weaker. Depth does not become weight, because the ranking is a trained disposition rather than an enforced precedence, and arriving tool output sits at the bottom of it.

Two composition rules

Observation is the intersection over gateways. Reach is the union over nodes. A chain is only as auditable as its least-observed edge, and no single gate evaluates the union.

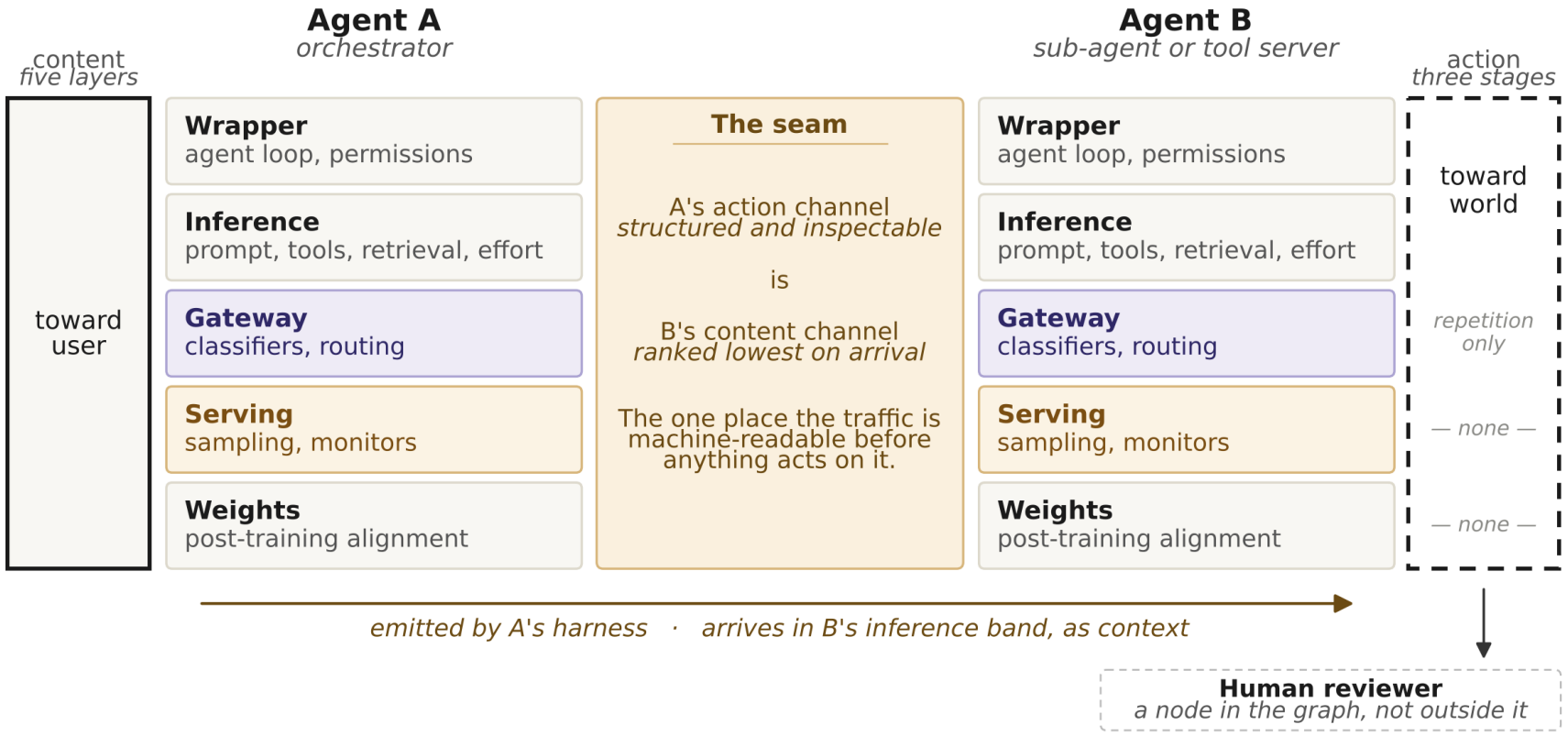
The seam is also the opportunity

A tool call is a named function and typed arguments, readable before anything acts on it. The prose either side is not. It is the only place a control can attach mechanically.

What is not claimed Branching, cycles, error amplification across hops, and agents sampling one model are all outside this. The rules are stated for a chain because that is where they are cleanest.

Between agents, the boundary becomes a seam

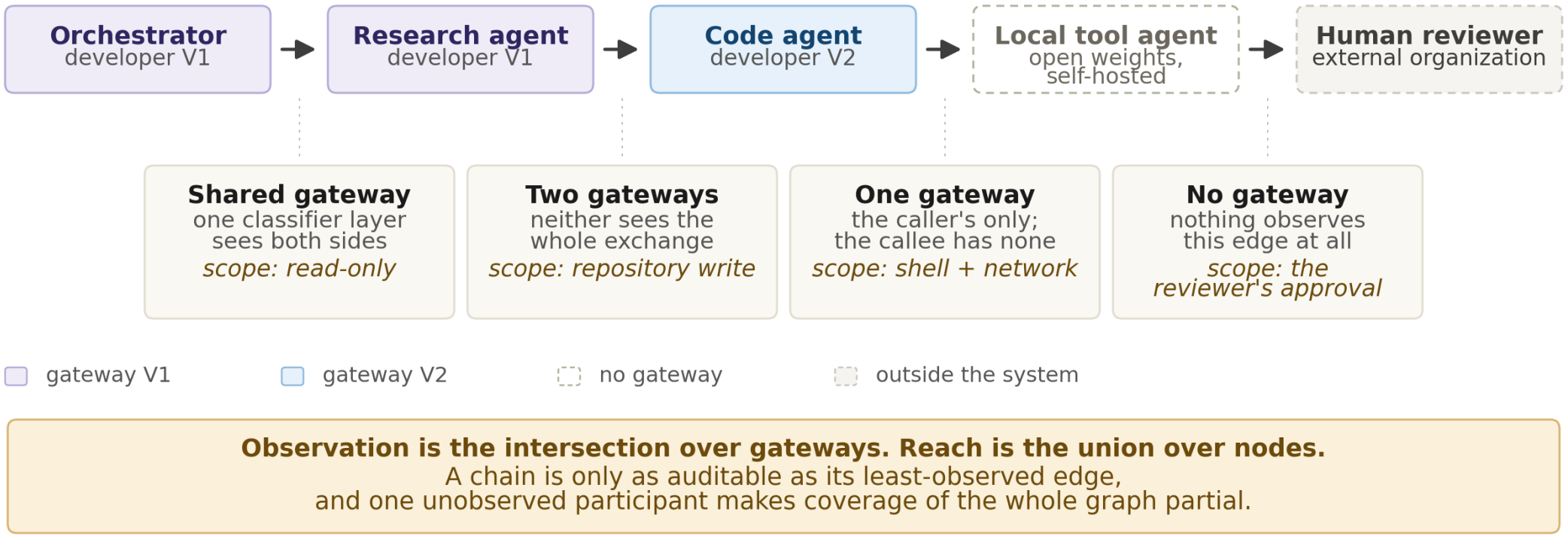
Departure controls and arrival controls, owned by different parties, neither auditing the other.



One edge, not a system Two agents are shown because that is the smallest case in which a seam exists — not because the framework treats multi-agent as meaning two.

Edges differ. Coverage is the weakest of them

The same chain with the framework's information moved onto the edges rather than the nodes.



Observation is the intersection over gateways. Reach is the union over nodes.
A chain is only as auditable as its least-observed edge,
and one unobserved participant makes coverage of the whole graph partial.

Why open weights matter here One self-hosted participant does not merely lack a gateway of its own. It makes gateway coverage of the entire graph partial, because the edges it touches are observed by nobody.

A second worked example: an embodied system

The workbench example exercises the content channel. This one exercises the action channel, which is where the framework is most at risk of not holding.

It is two stacks, not one

A reasoning model plans and calls tools; a separate action model moves things and is addressed as a declared tool. The reasoning model sits behind a hosted gateway. The on-device action model does not.

The last gateable point

The reasoning model emits structured tool calls. The action model emits numerical joint values — no named function, no typed argument. Mechanical inspectability dies at the handoff.

A stronger bottom stage than software has

Safety-rated stops, torque limits and light curtains are certified, hardware-enforced and independent of every band above. Nothing else in this framework can claim that.

A third reason coverage fails

Inside a closed control loop, per-request classification is infeasible at the rate the loop runs. Unlike developer fragmentation and open weights, a latency budget does not dissolve.

Read carefully Detecting proximity and triggering a safety stop describes a choice. A safety-rated stop is a constraint that holds when the model is wrong, absent, or adversarial. Different objects.

An embodied system does not add one band

It fills all of them, and one class of control sits outside the stack entirely.

Band	What occupies it in an embodied system	Which model
Product *domain layer*	Skills library, workcell configuration, allowed-object lists, task definitions	both · deployer's
Wrapper *swappable*	Motion planner, inverse-kinematics solver, controller loop, teleoperation handoff	both · deployer's
Inference *legible, live*	Camera and depth feed, proprioception, task prompt, the action model declared as a tool	reasoning model
Gateway *developer-side*	Present for the reasoning model through the hosted API. Absent for the on-device action model	reasoning model only
Serving *operator-side*	Hosted serving for the reasoning model; the on-robot runtime that runs the action model	both
Weights *baked-in*	Two weights bands in one system: the reasoning model and the action model	both
Outside the stack *certified, model-independent*	Safety-rated monitored stop, torque and force limits, e-stop, light curtains	neither

What the mapping produces The embodiment is the credential scope. And the inference band gains context that cannot be cleared: a transcript can be reset, a camera feed cannot.

What this framework cannot do

Five gaps, stated because a decomposition is more useful when it names where it stops than when it appears to work everywhere.

No process dimension

Nothing here says when in a lifecycle a band is built, reviewed, or certified — which is exactly what a conformity regime needs in order to act on any of it.

No capability gradation

The framework cannot yet say that at some capability level a particular band becomes mandatory. The bridge to threshold frameworks is unbuilt, and it is the obvious next section rather than a permanent gap.

Multi-agent structure, mostly not

Two claims only: an edge decomposes into a seam between two channels, and observation is the intersection over gateways while reach is the union over nodes. Branching, cycles, error amplification across hops and agents sampling one model are all outside it.

No magnitude for duration

The sixth question sorts the gate into what holds and what decays, but it supplies a direction and not a rate. At what run length reviewer attention stops being a control, and at what trajectory length a stated requirement stops influencing the next decision, are both unmeasured — and the bands have nowhere to put a number if they were measured.

And one that is deliberate No numbered levels or certifiable tiers. Premature quantification is how the most successful levels framework in engineering acquired a tier that functions mainly as marketing. The cost is that it limits uptake by anyone who has to write a rule.

What counts as too much, and who sets it

The bands say where a control can sit and who owns it. They say nothing about how much of anything is acceptable.

Why every band is described by how it fails

Unlike a network stack or a ring model, this diagram is not about how the parts interoperate. The bands above the weights exist to bound what the band below them does. Containment is the purpose, so the manner of failure is the property that matters, and a control that fails without announcing it compensates for nothing.

The threshold is not zero, and not uniform

Very wide availability of conventional-explosive chemistry is tolerated everywhere; almost none is tolerated for the pathogen classes under nucleic-acid screening. Not different in kind — different in how many people one success reaches, whether the harm stops on its own, and whether anyone can be compensated afterward.

The objective is contested

One regime is built around catastrophic misuse and systemic risk. Another is built around the state's account of what is true and what may be said, aimed at what citizens may be told rather than what could be built with a model's help. Both configure the same six bands. Neither needs a band the other lacks.

Under served access, the gateway is where this is sharpest

Invisible, non-removable, revisable without notice, on the path of every request. Those properties make it the only band that can hold a rule applying to everyone at once. They are also what would make it the most capable instrument of content control yet placed in a commercial supply chain.

The decision is not a technical one Nothing about a model determines what it should be prevented from doing. Whoever is positioned to decide does, for reasons running from irreversible harm through law, liability and product tone to a government's account of what may be said. The same control serves every one of them, and nothing in the control records which reason put it there.

What the diagram is for

01 Name the boundaries, not just the bands

A stack drawn only by provenance omits everything that governs reach — and reach is what an agentic product sells.

02 Ask how each band fails, not how strong it is

Bands differ less in strength than in failure signature. Silent failure is the property worth pricing.

03 Treat configuration as part of the architecture

A boundary whose integrity depends on a default is a weaker claim than the picture suggests.